

JAVA-FSE-Complete
MODULE 14
Agile Methodology

Aadi4-gif

Executive Summary

In modern software development, adaptability, rapid delivery, and customer collaboration are paramount. This module covers Agile Methodology, a dynamic framework designed to replace rigid, linear development models with iterative, incremental delivery. By emphasizing cross-functional teamwork, continuous feedback, and structured workflows like Scrum, Agile empowers organizations to respond swiftly to changing business needs and deliver high-value products efficiently.

Agile & Agile Manifesto

- The Agile Manifesto: Built upon four foundational values that prioritize individuals and interactions, working software, customer collaboration, and responding to change over comprehensive documentation and strict contract negotiation.
- 12 Agile Principles: Guide teams toward customer satisfaction through early and continuous delivery, welcoming changing requirements, maintaining sustainable development paces, and fostering motivated individuals.

- Agile vs. Waterfall: While Waterfall relies on sequential, rigid phases (requirements, design, implementation, testing, deployment), Agile breaks projects into iterative cycles (sprints), allowing for continuous testing, pivoting, and early value realization.

Scrum Framework

Scrum is one of the most popular frameworks for implementing Agile practices.

Scrum Roles:

- Product Owner (PO): Represents the stakeholder voice, manages and prioritizes the Product Backlog, and defines business value.
- Scrum Master: Acts as a servant-leader, facilitates Scrum ceremonies, removes impediments, and protects the team from external distractions.
- Development Team: Cross-functional group responsible for building, testing, and delivering the product increment.

Scrum Ceremonies:

- Sprint Planning: Collaboratively selecting backlog items for the upcoming sprint.
- Daily Scrum (Standup): A 15-minute synchronization meeting to track progress and identify blockers.
- Sprint Review: Demonstrating the completed work to stakeholders to gather feedback.
- Sprint Retrospective: An internal team reflection meeting to identify process improvements for the next cycle.

Scrum Artifacts & Definition of Done:

- Product Backlog: The master list of all desired work and features.
- Sprint Backlog: The subset of backlog items committed to for the current sprint.
- Increment: The usable, potentially releasable sum of all product backlog items completed during a sprint.
- Definition of Done (DoD): A shared agreement on the criteria that must be met before a work item is considered complete.

Agile Estimation & Planning Techniques

- **Story Points:** A relative unit of measure used to estimate the effort, complexity, and risk of implementing a user story, rather than tracking hours.
- **Planning Poker:** A collaborative, consensus-based estimation technique where team members secretly select story point cards and discuss discrepancies to align on estimates.
- **Sprint Planning & Velocity:** Velocity measures the amount of work (in story points) a team successfully completes during a single sprint, helping forecast future capacity.
- **Burndown Charts:** Graphical representations showing the amount of work remaining in a sprint over time, tracking progress toward the goal.

Agile User Stories & Acceptance Criteria

- User Story Format: Captures requirements from an end-user perspective using the standard syntax:
- As a [type of user], I want [some goal], so that [some benefit].
- The INVEST Principle: Ensures high-quality user stories that are Independent, Negotiable, Valuable, Estimable, Small, and Testable.
- Acceptance Criteria (Given-When-Then): Defines the specific boundaries and conditions for a user story to be accepted, often structured using Behavior-Driven Development (BDD) syntax:
 - Given: The initial context or precondition.
 - When: The specific action or trigger occurs.
 - Then: The expected outcome or result.

Conclusion

Agile methodology transforms how software teams plan, build, and deliver products. By embracing Scrum ceremonies, relative estimation techniques, disciplined user story creation, and continuous feedback loops, organizations can boost team morale, enhance adaptability, and consistently deliver high-value software that aligns closely with user needs.